Table of Contents

Sprint 1

Sprint Report and Requirement Specification

Project overview (In order of operations a user would realistically face using the app)

1) This project allows users to select and rent cars for a chosen period at a fixed daily rate. This is done by first asking the user to sign in with an existing account or to create a new account.

- If the user creates a new account, they must enter their:
    - Username
    - Age
    - Password

    - Driver's license number
    - Driver's license expiry date
    - Select Membership type
- Standard checks when creating an account
    - No fields are left blank
    - Username doesn't already exist
    - User is 21 years or older
    - Password is 8 or more characters
    - Driver's license is valid
    - Driver's license cannot already be taken
    - Membership type is selected
        - Standard Users:
            - No discounts
            - no late returns allowed (immediate penalty)
            - Late return rate of 50% per overdue day
            - Can rent only 1 vehicle at a time
        - Premium
            - 10% discount on daily rate of rentals
            - 1 day grace period on late returns
            - Late return rate of 25% per overdue day
            - Can rent 2 vehicles at a time

2) Once a user has created an account they can then login.

- Standard checks when logging-in to an account
    - No fields are empty
    - Valid username and password combination exist to log user in

- Once a user is signed in, they can:
    - View vehicles available
    - Vehicles they have rented
    - Their account
    - Their cart to rent vehicles or settle penalties

3) If the user chooses to rent a vehicle he will be presented with the option of renting 4 types of vehicles with varying daily rates

Vehicle Type	Daily Rate
Economy	$50
Sedan	$70
Luxury	$150
SUV	$90

NOTE: Luxury vehicles can only be rented by users who are 25 years or older (there is a check in place for this when the user is viewing the vehicles).

4) Once the user has selected the vehicle and the number of days they want to rent the vehicle for, they can add this vehicle to the cart.

- What the cart displays
    - The name of the vehicle in the cart
    - The number of days the vehicle will be rented for
    - The total price to rent the vehicle for the duration before other charges
    - Any discounts the user might be applicable for (EX: Premium user 10% discount)
    - Any surcharges the user might be applicable for (EX: young renters' fee: <25 years old)
    - Final total after all charges are applied
    - Option to remove vehicle from cart
    - Any outstanding penalties that need to be paid
- Checks performed on the cart
    - The cart is not empty
    - A user session exists when adding vehicles to the cart
    - Drives license has not expired
    - No more than 1 or 2 rentals at a time depending on user type.
    - Users under 25 years old cannot rent luxury vehicles
    - Vehicle isn't already rented
    - Vehicle is being rented for at least 1 day
    - User has enough money in account to rent vehicles.

5) If the user does not have enough money in their account, they can manually add money by entering an amount. A simple check is done to ensure a negative amount isn't entered.
6) Once vehicles have been rented these vehicles are marked as unavailable to all other users and users can view vehicles they have rented in My Rentals

- Information displayed in My Rentals
  - Vehicle name
  - Date vehicle was rented on
  - When rental is due to be returned
  - Is the rental still active
  - Option to return rental

7) Finally, users can return vehicles that have been rented from the My Rentals section.

- Checks done on returning a vehicle
  - Vehicle hasn't already been returned
  - Vehicle is not overdue by more than 2 hours of return window
  - If overdue by more than 2 hours mark as late return
    - Premium users:
      - get 1-day late penalty grace period
      - 25% late return charge per overdue day
    - Standard users
      - No grace period
      - 50% late return charge per overdue day
  - Apply late return penalty per overdue date depending on user type
  - Send total to cart to be paid by user

Use Cases

Field	**Description**
Use Case ID	UC-01
Use Case Name	Rent a Car

Actors	Registered Member (Standard or Premium), System
Preconditions	User must have a valid account (logged in) User has a valid driver's license User meets minimum age requirements Vehicles exist in the system with available status
Trigger	User selects a vehicle class and chooses "Rent"
Main Flow	1. User selects desired vehicle class (Economy, Sedan, SUV, Luxury 2. System checks age eligibility and license validity. 3. System checks vehicle availability 4. If available, system assigns a return date/time 5. System calculates rental cost 6. System marks vehicle as unavailable 7. Rental record is created and stored
Alternative Flows	A1 at step 1- Age Restriction - If user is of age 21-24 Luxury vehicle is unavailable A2 at step 2 - Invalid License - if license is expired or invalid, rejects with "Driver's license not valid"
Postconditions	Success: Vehicle is reserved, user has a rental record, and cost is calculated Failure: User remains on rental page with rejection message
Business Rules / Notes	Rentals are charged in 24-hour increments Premium members receive 10% discount and can rent up to two cars at once Standard members can rent only one vehicle.

Field	Description
Use Case ID	UC-02
Use Case Name	Return a Car
Actors	Registered Member, System
Preconditions	User has an active rental
Trigger	User returns vehicle before, at or after the scheduled return time
Main Flow	1. System calculates rental duration 2. System checks for late return 3. System calculates any mileage overage charges 4. Total cost = base rental + late fees + mileage overage 5. Vehicle is marked available 6. Rental record is closed
Alternative Flows	A1 at step 2 - If late: Standard Member: Late fee = 50% of daily rate × days late Premium Member: One-day grace. After grace: 25% of daily rate × days late.
Postconditions	Success: Vehicle is returned, available for future rentals, charges applied Failure: Rental record remains open
Business Rules / Notes	Late = returned more than 2 hours past scheduled time Mileage cap: 200 miles/day Extra miles = $0.25/mile

Field	**Description**

Use Case ID	UC-03
Use Case Name	View Rented Vehicles
Actors	Registered Member (Standard or Premium), System
Preconditions	User must have a valid account (logged in) User must have already rented a vehicle Vehicles exist in the system and is marked as rented
Trigger	User selects “My Rentals” on their account
Main Flow	1. User views vehicles they have rented on “My Rentals” page 2. Page displays vehicle name, rented date, return date and status to user 3. User is given the option to return the rental
Alternative Flows	- None
Postconditions	UC-02 (Return a car) is invoked
Business Rules / Notes	- Same as UC-02

Field	**Description**
Use Case ID	UC-04
Use Case Name	View Login page
Actors	New users, Existing members, System
Preconditions	None
Trigger	User clicks Sign in button

Main Flow	- System returns login.html - System provides login form
Alternative Flows	- If form already exists keep existing instance
Postconditions	Login page is displayed with form

Field	**Description**
Use Case ID	UC-05
Use Case Name	Submit login
Actors	Registered user
Preconditions	- Login page is displayed - Username and password is provided
Trigger	User clicks log in button
Main Flow	- System validates input - System authenticates user - For valid username and password log user in
Alternative Flows	- Validation errors such as blanks fields displays errors - Wrong credentials reject and stay on login page
Postconditions	- Success login creates user and displays user dashboard - Invalid login remains on login page
Business Rules / Notes	Authenticate and validate user credentials before logging them in

Field	**Description**

Use Case ID	UC-06
Use Case Name	Logout
Actors	Registered user
Preconditions	User must be logged in
Trigger	User clicks sign-out button
Main Flow	- System invalidates session - User redirected to homepage
Alternative Flows	- None
Postconditions	User session removed

Field	**Description**
Use Case ID	UC-07
Use Case Name	View registration page
Actors	New user, members, system
Preconditions	
Trigger	User clicks Register button
Main Flow	- System loads page - Addes new registration form if its not present - Loads membership type options
Alternative Flows	- Existing form is retained and displayed
Postconditions	Registration page displayed with complete from

Field	**Description**

Use Case ID	UC-08
Use Case Name	Submit registration
Actors	New users
Preconditions	Registration page is displayed
Trigger	User clicks Create Account button
Main Flow	- Validate information and display errors - Register user - If successful show success message
Alternative Flows	- Validation errors display errors and stay on page - Duplicate usernames display errors and stay on page - Duplicate license display errors and stay on page
Postconditions	- Successful registration displays success page - Otherwise remain on page with errors

Field	**Description**
Use Case ID	UC-09
Use Case Name	Get user account
Actors	Registered User
Preconditions	Account exists in repository
Trigger	System requests account details
Main Flow	- User clicks account button to view account balance - System calls instance of user's account - User can view account
Alternative Flows	- If user tries to view account details without being logged in - Thrown UserNotFoundException

Postconditions	- Account returned or error thrown

Field	**Description**
Use Case ID	UC-10
Use Case Name	Add funds to wallet
Actors	Registered user
Preconditions	User is logged in and on the account page
Trigger	User enters an amount and clicks “Add Funds”
Main Flow	- System calls addFunds and passes userID and amount - Amount it added to wallet and saved
Alternative Flows	- User enters amount that is <= 0 and an exception is thrown
Postconditions	- Successfully add funds to page - Otherwise throw error

Field	**Description**
Use Case ID	UC-11
Use Case Name	Debit wallet balance
Actors	Registered user
Preconditions	User is logged in and has an item in cart ready to checkout with enough funds in their account
Trigger	User clicks checkout
Main Flow	- System debits amount from user account - New amount in account is saved

Alternative Flows	- If amount is > balance, then thrown error
Postconditions	- Successfully deduct funds or throw error

Sprint 1 Report

Overall, this sprint was successful in covering required criteria for project milestone 1 within the given period. For this project the team implemented:

- A robust back end with business logic containing criteria specific to 2 types of users on the app
- A very simple but functional UI with safeguards to help mitigate app crashes
- 4 unique types of rentals which are controlled by age checks
- Discount logic for premium users
- Surcharges for young renters
- Calculated return windows by dates and times
- Late return grace period for premium users
- Varying late return rates based on user type
- Checking finances before processing transactions

Features and Areas covered by each team member

Hiruna Yevin Dissanayake

- Application Setup
- Login
- Registration
- User authentication and sessions
- Cart and checkout logic

Tyson Russell

- Vehicles.html page
- Vehicle constructors
- Vehicles data entry
- Rental service

Ivan Quintero

- Setting up vehicle and rental entities
- Business Logic in backend
    - Vehicle checkout
    - Vehicle return
    - Calculating late fees
    - Calculating additional fees if applicable
    - Calculating total charge

SPRINT 2

Features covered by each team member

Tyson Russell

- Pricing service tests
- Cart service tests
- Cart Controller tests

Hiruna Yevin Dissanayake

- Code refactored to move logic out of controllers
- Account service tests
- Authentication tests
- Account controller tests
- Authentication controller tests
- Global exception handler and home controller tests

Ivan Quintero

- Car rental service tests
- Vehicle service tests

Test Cases

Log in Test Cases

ID	Test case names	Description	Preconditions	Test steps	Expected results	Test data

UC-05 TC-01	Authenticate Success Returns User	Ensure valid user can login successfully	Valid user account already exists	Call service.authenticate("Test", "ok");	User object is returned that matches user in repo	Username: Test Password: ok
UC-05 TC-02	Invalid login UsernameNotFoundThrowsException	Verify username that doesn't exist thrown error	No account "Test"	Call service.authenticate("Test","pwd")	Throws error for username	Username: Test Password: pwd
UC-05 TC-03	Invalid login WrongPasswordThrows Exception	Verify wrong password thrown error	User "Test" exists but password mismatch	Call service.authenticate("Test","bad")	Throws exception for password	Username: Test Password: bad

Registration Test Cases

ID	Test case names	Description	Preconditions	Test Steps	Expected Results	Test Data
UC-08TC-04	Registration password hash saves UserPasswordHashesAndSaves	Verify password is hashed after user is saved	Repo saves user with	Call service.register(u, "password");	Hahsed passwrod is "ENC"	Password: password
UC-	registerDuplicateUsernameThrowsException	Reject duplicate	existsByUsername("u1") = true	Submit registration form	Throws DuplicateUsernameException	Username: u1

08TC-05		usernames				
UC-08TC-06	registerDuplicateDriversLicenseThrowsException	Reject duplicate driver's license numbers	Username free and existsByDriversLicense("DL123") = true	Submit registration from	Throws DriversLicenseTakenException;	Drivers License number: DL123
UC-08TC-07	registerUserWithNoMembershipDefaultsToStandard	Verify system sets user as STANDARD if none is selected	Username and drivers license are valid	Submit registration form with membershipType = null	User saves with STANDARD	Username: u1 Password: pw-12345678
UC-08TC-08	registerUserWithMembershipRetainsMembershipType	Verify that membership selected is saved	Username and driver's license is valid and membership is selected	Submit form with membershipType = PREMIUM	User saves with PREMIUM	Membership type: PREMIUM

Account Service Test cases

ID	Test case names	Description	Preconditions	Test steps	Expected results	Test data
UC-09TC-01	testGetByIdFound	Ensure existing users account is returned by ID	User with ID exists	Call accountService.getById(1L)	Returns user from repo	ID:1
UC-09TC-02	testGetByIdNotFound	Check error is thrown if account for nonexistent ID is called	ID being called does not exist	Call accountService.getById(2L)	Throws UserNotFoundException	ID:2
UC-09TC-03	testGetByUsernameFound	Check account with existing username is returned	User with username exists	Call accountService.getByUsername("tytyruss")	Returns users account	Username: tytyruss
UC-09	testGetByUsernameNotFound	Check account call for nonex	Usernae doesn't exist	UserNotFoundExcepti on	Throws UserNotFoundException	Username: DNE

TC-04		istent username thrown error				
UC–10TC-05	testAddFundsPositive	Ensure funds with positive value get added to account and saved	User exists with account and balance 100	Call accountService.addFunds(1L, 50.0)	Balance increases to 150	ID: 1 Amount: 50
UC-10TC-06	testAddFundsZeroOrNegative	Adding an amount <=0 throws error	-	Call addFunds(1L, 0) and addFunds(1L, -10)	Throws exception and doesn't make changes to account	ID: 1 Amounts: 0, -10
UC-11TC-07	testDebitBalance	Ensure debit reduces balance and saves	User with account exists and balance 100 in account	Call accountService.debit(1L, 50.0)	Balance drops to 50	ID:1 Amount:50
UC-	testDebitExactBalance	Ensure exact	User with accou	Call accountService.debit(1L, 100.0)	Balance drops to 0.0	D: 1 Amou

11TC-08		amount in account debited leaves 0 in account	nt exists and balance 100 in account			nt: 100.0
UC-11TC-09	testDebitInsufficientFunds	Ensure trying to draw more than what is in the account throws exception	User with account exists and balance 100 in account	Call accountService.debit(1L, 150.0)	Throws InsufficientFundsException	ID: 1 Amount: 150.0

Cart Service Test

ID	Test Case Name	Description	Preconditions	Test Steps	Expected Results	Test Data
UC-01-TC-01	testAddRentLineStandardUserAdult	Verify that a standard user can add a rental item correctly	User is logged in	Call addRentLine()	Cart contains 1 item with: Type = RENT Base amount = 150.0 Discount = 0.0 Surcharge = 0.0	Membership Type=STANDARD Age=30 DailyRate=50.0 Days=3

					Total amount = 150.0	
UC-02-TC-02	testAddRentLinePremiumUserUnder25	Verify premium users under 25 get both discount and surcharge adjustments	User is logged in	Call addRentLine	Cart contains 1 item with: Base amount = 200.0 Discount = 20.0 Surcharge = 4.0 Total amount = 184.0	MembershipType=PREMIUM Age=22 DailyRate=50.0 Days=4
UC-02-TC-03	testAddPenaltyLine	Verify that penalty is added to cart	User has existing rental record	Create rental Call addPenaltyLine	Cart contains 1 item with: Type = PENALTY Amount = 25.0	RentalID=1 PenaltyAmount=25.0
UC-01-TC-04	testTotalWithItems	Verify total amount correctly	User logged in, cart contains multiple items	Create cart with items Call total(cart)	Returns 75.0	Amounts=[50.0,25.0,null]

Rental Service Test

ID	Test Case Name	Description	Preconditions	Test Steps	Expected Results	Test Data
UC-01-T	test_createRental_zeroDays	Verify rental fails if	N/A	Call rentalService.createRental() with 0 days	IllegalArgumentException is thrown with message “Days	days = 0

C-05		days are 0.			must be >= 1".	
UC-01-TC-06	test_createRental_negativeDays	Verify rental fails if days are negative.	N/A	Call rentalService.createRental() with -1 days	IllegalArgumentException is thrown with message "Days must be >= 1".	days = -1
UC-01-TC-07	test_createRental_userNotFound	Verify rental fails if the user ID does not exist.	appUserRepository.findById() is mocked to return Optional.empty()	Call rentalService.createRental() with a non-existent user ID.	UserNotFoundException is thrown.	userId = 99
UC-01-TC-	test_createRental_vehicleNotFound	Verify rental fails if the vehicle	appUserRepository.findById() returns a valid user.	Call rentalService.createRental() with a non-existent vehicle ID.	VehicleNotFoundException is thrown.	vehicleId = 999

08		ID does not exist.				
UC-01-TC-09	test_createRental_vehicleUnavailable	Verify rental fails if the vehicle isAvailable() is false.	appUserRepository.findById() returns a valid user. vehicleRepository.findById() returns a vehicle with available = false	Call rentalService.createRental() with an unavailable vehicle ID.	VehicleUnavailableException is thrown.	vehicleId = 102
UC-01-TC-10	test_createRental_success	Verify the "happy path" for a successful rental. (Covers UC-01	All mocks return valid data (user, available vehicle). pricingService.rentalTotal() returns a valid price	Call rentalService.createRental().	A new Rental object is created and saved with correct dates, price, user, and vehicle. The vehicle object is saved with available	userId = 1 vehicleId = 101 days = 5

		Main Flow steps 5, 6, 7)			set to false.	
UC-01-TC-11	test_createRental_daysOverload_OneDayIfNull	Verify the Integer days overload defaults to 1 day if null is passed	All mocks return valid data.	Call rentalService.createRental() with days = null.	The service calls pricingService.rentalTotal() with days = 1	days = null
UC-03-TC-01	findAllForUser_shouldReturnRentals	Verify a user can retrieve their list of	rentalRepository.findByUserIdOrderByRentedAtDesc() is mocked to return a list of rentals.	Call rentalService.findAllForUser().	The service returns the exact list of rentals provided by the repository.	userId = 1

		rentals. (Covers UC-03 Main Flow)				
UC-02-TC-04	markReturned_shouldDoNothing_whenAlreadyReturned	Verify that returning an already-returned rental does nothing. (Covers UC-02 Alternative Flow)	Rental object has a non-null returnedAt date.	Call rentalService.markReturned().	No save() methods are called on any repository.	Rental with returnedAt = [date]

UC-02-TC-05	markReturned_shouldUpdateRentalAndVehicle_whenNotReturnedAndVehicleUnavailable	Verify a successful return updates rental status and vehicle availability. (Covers UC-02 Main Flow steps 5, 6)	Rental has returnedAt = null. Vehicle has available = false.	Call rentalService.markReturned().	RentalRepository.save() is called. Saved rental has returnedAt set to today. VehicleRepository.save() is called. Saved vehicle has available = true.	Rental with returnedAt = null.
UC-02-TC-	markReturned_shouldUpdateRentalOnly_whenNotReturnedAndVehicleAlreadyAvailable	Verify service handles	Rental has returnedAt = null. Vehicle has available = true.	Call rentalService.markReturned().	RentalRepository.save() is called. VehicleRepository.save() is	Rental with returnedAt

06		data anomaly where vehicle is already marked available.			not called.	= null.

Vehicle Service Test

ID	Test Case Name	Description	Preconditions	Test Steps	Expected Results	Test Data
UC-01-TC-01	list_shouldReturnAllVehicles_whenAvailableIsNull	Verify that listing vehicles with a null filter returns all vehicles.	vehicleRepository.findAll() is mocked to return a list of 2 vehicles.	Call vehicleService.list(null)	The service returns the complete list of 2 vehicles. findAll() is called.	available = null
UC-01	list_shouldReturnOnlyAvailableVehicles_whenAvailableIsTrue	Verify that the available =	vehicleRepository.findByAvailable(true) is mocked to	Call vehicleService.list(true)	The service returns the filtered list of 1	Available = true.

-TC-02		true filter returns only available vehicles.	return a list of 1 vehicle.		vehicle. FindByAvailable(true) is called.	
UC-01-TC-03	list_shouldReturnOnlyUnavailableVehicles_whenAvailableIsFalse	Verify that the available=false filter returns only unavailable (rented) vehicles.	vehicleRepository.findByAvailable(false) is mocked to return a list of 1 vehicle.	Call vehicleService.list(false)	The service returns the filtered list of 1 vehicle. FindByAvailable(false) is called.	Available = true.
UC-01-TC-04	get_shouldReturnVehicle_whenFound	Verify the service can retrieve a single vehicle by its ID.	VehicleRepository.findById(1L) is mocked to return Optional.of(vehicle1).	Call vehicleService.get(1L).	The service returns the correct vehicle1 object.	vehiclelId = 1L
UC-01-TC-	get_shouldThrowVehicleNotFoundException_whenNotFound	Verify that requesting a non-existent	VehicleRepository.findById(99L) is mocked to return Optional.Empty()	Call vehicleService.get(99L).	VehicleNotFoundException is thrown.	vehiclkeId = 99L

07		vehicle ID throws an exception.				

Test Outcomes

The team did not find any bugs pertaining to the services of the application. When refactoring as per feedback of sprint 1 all major aspects of the application were taken into consideration. Therefore, resulting in a successful test suite pass from the very biggening. These tests were then refined over each iteration to ensure complete branch coverage of all services.

Test Plan

Integration Testing Plan

Our objective is to verify that individual modules developed by different team members interact correctly with one another. This ensures that data flows, validations, and session handling across modules are handled correctly. Integration testing will cover the following module interactions: Registration and Login, Login and Vehicle listing. Cart and Vehicle listing, Checkout and My rentals, Rentals and Returns. We will begin working from top down, starting with login and down.

System Testing Plan

Our objective is to verify that the application meets all functional and business requirements. We will use block box techniques to verify that the system behaves as specified. The following areas will be tested: Registration, Login/logout, Vehicle listings, Cart, Checkout, My rentals, Rent/Return vehicles. We plan to use Selenium for functional UI tests.

Performance Testing Plan

Our objective is to assess the responsiveness, scalability, and stability of the system under normal and peak loads. We will test the following metrics: Response time, Throughput, Database query time, Scalability. We expect to learn some tools for performance testing in the coming weeks in lecture. Some of the tools may include Apache JMeter or k6.